

PROGRAMACIÓN IV

Trabajo Práctico N° 1

Desarrollo de una aplicación web con Ruby on Rails

Proyecto individual

Propósito del trabajo

Diseñar y construir una aplicación web completa aplicando los principales conceptos de Ruby on Rails, desde el modelado del dominio y el back-office hasta una API autenticada, testing, calidad de código y seguridad.

1. Introducción

Cada estudiante deberá elegir libremente la temática de la aplicación. La propuesta debe representar un problema o necesidad concreta y permitir desarrollar un dominio suficientemente rico como para aplicar los conceptos trabajados durante la materia.

Algunas temáticas posibles:

- Gestión de biblioteca.
- Plataforma de cursos.
- Sistema de reservas o turnos.
- Tienda de productos.
- Gestión de eventos.
- Plataforma de recetas.
- Sistema de publicaciones o noticias.
- Plataforma deportiva.

Antes de comenzar el desarrollo, cada estudiante deberá definir brevemente:

- El objetivo general de la aplicación.
- Los tipos de usuario que utilizarán el sistema.
- Las entidades principales y sus relaciones.

2. Objetivos

En este trabajo se desarrollará el back-end de una aplicación que luego integrará con el front end a desarrollar en el trabajo práctico N°2. Otro de los objetivos es integrar los principales conceptos del desarrollo de aplicaciones que se pueden explorar a través de Ruby on Rails:

- Arquitectura MVC y routing.
- Controladores, vistas y formularios.
- Active Record, migraciones, validaciones y asociaciones.
- Operaciones CRUD.
- Autenticación y autorización.
- Desarrollo de APIs JSON.
- Active Storage y Action Mailer.
- Testing automatizado.
- Calidad de código y análisis de seguridad.
- Uso profesional de Git y documentación del proyecto.

3. Requerimientos obligatorios

3.1. Tecnología y repositorio

La aplicación deberá estar desarrollada utilizando Ruby on Rails. El proyecto deberá utilizar Git para el control de versiones y encontrarse alojado en un repositorio accesible.

Los commits deberán reflejar razonablemente la evolución del proyecto.

3.2. Modelo de datos

La aplicación deberá contar con al menos cinco modelos principales relacionados entre sí. Los modelos deberán representar entidades relevantes del dominio elegido y no existir únicamente para alcanzar la cantidad mínima solicitada.

Los modelos deberán incluir las validaciones necesarias para mantener la consistencia de los datos.

3.3. Back-office

La aplicación deberá incluir una sección de administración o back-office desarrollada con las herramientas tradicionales de Rails: controllers, views, templates, formularios, routing y Active Record.

El back-office deberá permitir administrar las principales entidades mediante operaciones CRUD y deberá estar protegido mediante autenticación. Las funcionalidades administrativas solamente deberán estar disponibles para usuarios autorizados.

Se recomienda organizar esta parte utilizando un namespace específico:

```
namespace :admin do
  resources :products
  resources :categories
  resources :orders
end
```

Ejemplos de URLs administrativas:

```
/admin/products
/admin/categories
/admin/orders
```

3.4. API

Además del back-office, la aplicación deberá exponer una API destinada a ser consumida por el front-end público del sitio. Las respuestas deberán utilizar JSON y la API deberá encontrarse separada conceptualmente de los controladores utilizados por el back-office.

Se recomienda utilizar versionado de endpoints, por ejemplo:

```
GET    /api/v1/products
GET    /api/v1/products/:id
POST   /api/v1/orders
GET    /api/v1/profile
```

3.5. Autenticación de usuarios

La aplicación deberá contemplar, como mínimo, dos contextos de utilización:

Usuario administrativo	Usuario final
Accede al back-office y a las operaciones de administración del sistema. Se puede utilizar el sistema de autenticación que viene integrado en Rails.	Consume el front-end público y deberá poder autenticarse contra la API. Por ejemplo con un endpoint donde se mande usuario y contraseña, y la API conteste con un token que será utilizado para posteriores requests que necesiten autenticación.

No necesariamente ambos tipos de usuarios deberán estar representados por modelos diferentes. La solución dependerá del diseño elegido por cada grupo.

3.6. Active Storage

El proyecto deberá incluir al menos un caso de uso real de Active Storage y deberá tener sentido dentro de la temática seleccionada (por ejemplo: imagen de un producto o publicación, avatar de usuario, material o documentación adjunta. archivos relacionados con una reserva, curso o trámite, etc)

3.7. Action Mailer

La aplicación deberá implementar al menos un envío de email mediante Action Mailer, relacionado con una acción real del sistema (por ejemplo: email de bienvenida, confirmación de reserva o compra, recuperación de contraseña. etc)

3.8. Testing

El proyecto deberá incluir tests automatizados. No se requiere una cobertura completa, pero como mínimo deberán existir pruebas relacionadas con modelos y validaciones y alguna lógica de negocio.

3.9. Calidad de código - Rubocop

El proyecto deberá utilizar Rubocop para analizar la calidad y el estilo del código Ruby. Antes de la entrega deberá ejecutarse y pasar la validación de Rubocop.

Si se decide ignorar alguna regla, deberá existir una justificación o configuración explícita.

3.10. Seguridad - Brakeman

El proyecto deberá utilizar Brakeman para realizar un análisis estático de seguridad de la aplicación Rails. Antes de la entrega deberá ejecutarse y pasar la validación de Brakeman.

No se exige necesariamente obtener cero advertencias, pero deberá tener justificación si se ignora alguna advertencia.

4. Funcionalidades opcionales

Como funcionalidad adicional, el grupo podrá implementar características más avanzadas del ecosistema Rails. La implementación correcta será considerada como un extra del trabajo.

Swagger

Documentación de los endpoints de la API de forma interactiva dentro de la aplicación.

Active Job

Tareas en segundo plano: envío de emails, procesamiento de archivos, generación de reportes o procesos costosos.

Action Cable (Web socket)

Comunicación en tiempo real: chat, notificaciones o actualizaciones de estado.

Hotwire / Turbo

Interactividad en las vistas Rails reduciendo la necesidad de JavaScript manual.

5. Deploy - opcional recomendado

De ser posible, la aplicación deberá encontrarse publicada en algún servicio que permita acceder a ella desde Internet utilizando un plan gratuito o educativo disponible al momento de realizar el trabajo.

6. Estructura sugerida del proyecto

Se recomienda separar claramente las responsabilidades del back-office y de la API. La siguiente estructura es únicamente una referencia y podrá adaptarse al dominio elegido:

```

app/
├── controllers/
│   ├── admin/
│   │   ├── products_controller.rb
│   │   ├── categories_controller.rb
│   │   └── orders_controller.rb
│   ├── api/
│   │   └── v1/
│   │       ├── sessions_controller.rb
│   │       ├── products_controller.rb
│   │       ├── orders_controller.rb
│   │       └── users_controller.rb
│   └── application_controller.rb
├── models/
│   ├── user.rb
│   ├── product.rb
│   ├── category.rb
│   ├── order.rb
│   └── order_item.rb
├── views/
│   ├── admin/
│   ├── layouts/
│   └── mailers/
├── mailers/
├── jobs/
└── services/

```

Las migraciones se encontrarán en db/migrate/ y las rutas principales en config/routes.rb.

Una posible organización de rutas sería:

```

Rails.application.routes.draw do
  namespace :admin do
    resources :products
    resources :categories
    resources :orders
  end

  namespace :api do
    namespace :v1 do
      resources :products, only: [:index, :show]
      resources :orders, only: [:index, :show, :create]
      post 'login', to: 'sessions#create'
      delete 'logout', to: 'sessions#destroy'
    end
  end
end

```

7. Organización sugerida del desarrollo

Etapas 1 - Definición

Temática, problema, tipos de usuario, funcionalidades principales, modelos y relaciones. Se recomienda realizar un pequeño diagrama de clases.

Etapas 2 - Modelos y base de datos

Crear migraciones, modelos, relaciones y validaciones. Antes de continuar debería ser posible crear y consultar los objetos principales desde rails console.

Etapas 3 - Back-office

Implementar autenticación administrativa, layout, navegación y CRUD de las principales entidades.

Etapas 4 - API

Implementar autenticación de usuario final, endpoints públicos y protegidos, respuestas JSON y manejo apropiado de errores HTTP.

Etapas 5 - Funcionalidades adicionales

Incorporar Active Storage, Action Mailer y tests. Opcionalmente, Active Job, Action Cable, Hotwire/Turbo o Swagger.

Etapas 6 - Calidad y seguridad

Ejecutar Rubocop y Brakeman; corregir o analizar los problemas encontrados.

Etapas 7 - Deploy

Si es posible, desplegar la aplicación y verificar el funcionamiento fuera del ambiente local.

8. Entrega

La entrega deberá incluir repositorio Git del proyecto que incluya un archivo README.md. Deberá permitir que otra persona pueda clonar el repositorio y ejecutar el proyecto sin necesidad de conocer previamente cómo fue desarrollado, incluyendo:

- Instrucciones para instalar y ejecutar el proyecto.
- Instrucciones para preparar la base de datos.
- Credenciales o mecanismo para acceder al back-office.
- Descripción de los endpoints principales de la API.
- Descripción breve del modelo de datos.
- URL del proyecto desplegado, en caso de haber realizado deploy.

Fecha de checkpoint: 16 de septiembre

Fecha de entrega para revisión: 30 de septiembre